MeetingAI - The Complete Project Guide

An AI assistant that listens to your meetings, remembers everything, and turns talk into action.

This guide explains the whole project in plain language - from "what is it?" to "how do I put it on the internet?". You do not need to be a programmer to follow the first parts. Technical parts are explained with simple examples and a glossary at the end.

Table of contents

1 What is MeetingAI?

2 The problem it solves

3 A quick tour of the app (every page)

4 How to use it, step by step

5 How it works inside - the story of one meeting

6 The technologies we used and why we chose each one

7 The NLP part explained simply

8 The "confidence gateway" - the heart of the project

9 How MeetingAI avoids making things up

10 Search and the chatbot - how they find answers

11 Where your data lives (and privacy)

12 The project folder, file by file

13 Setting it up on your own computer

14 Getting the free API keys

15 Putting it on the internet (Streamlit Cloud) - step by step

16 Other free places to host it

17 How we tested it, and what its limits are

18 Troubleshooting

19 Ideas for the future

20 Glossary

---

1. What is MeetingAI?

MeetingAI is a web application. You give it a meeting - a **recording** (audio or video) or a **written transcript** - and it:

- turns speech into text (a **transcript**) with timestamps, and shows **who said what by name** ("Priya: ...") using pyannote speaker recognition,
- writes a **summary** (overview, key points, decisions, conclusions),
- finds the **action items** - who must do what, and by when,
- finds **decisions** that were made,
- finds **events and dates** ("let's meet next Friday at 3 PM") and puts them on a **calendar**,

- lets you **chat** with your meetings ("Who is responsible for the report?"),
- lets you **search** across all meetings,
- writes professional **Minutes of Meeting (MoM)**, which you can edit, download, and **email**,
- suggests **follow-up messages and reminders** - but never sends anything without your say-so.

Think of it as a very organised assistant who attended every meeting, took perfect notes, and never forgets.

2. The problem it solves

In most meetings, important things get lost:

- "Who was going to send that report?" - nobody remembers.
- "What deadline did we agree on?" - three different answers.
- "Did we decide to hire the agency?" - someone has to re-watch an hour of video.

Writing minutes by hand takes time, and normal transcription tools only give you a wall of text. MeetingAI goes further: it **understands** the conversation, and **keeps everything organised and searchable**.

A very important design rule: **when MeetingAI does not know something, it says so.** If nobody was named as the owner of a task, the owner is shown as **"Unassigned"**. If no date was mentioned, the deadline shows **"Not specified"**. It never invents names or dates to look clever.

3. A quick tour of the app

The left sidebar has all the pages.

Page	What it does
Dashboard	Your overview: number of meetings, tasks done and pending, overdue tasks, upcoming deadlines and events, recent meetings, charts, and the health of the NLP engine.
New Meeting	Upload audio/video/transcript files (or paste a transcript) and watch it get analysed.
Meeting Library	All saved meetings as cards. Search, sort, open, ask questions, or delete.
Meeting Details	Opened from the library. Tabs: *Overview* (summary, key points, topics, participants), *Transcript* (speaker colours, timestamps, search, audio player, and the **Who is who?** panel for naming speakers), *Action items* (editable table), *Decisions & events*, and *How it was analysed* (why each item was trusted or double-checked).
Ask MeetingAI	The chatbot. Ask about one meeting or all meetings. Answers show their sources. Chat history is saved so you can continue later.
Search	Find anything across all meetings: smart (by meaning), keyword, or exact phrase. Shows meeting name, timestamp and the supporting text.
Action Tracker	One central table of tasks from every meeting, with owner, deadline, status (Pending / In Progress / Completed), and source meeting. You can edit everything.
Calendar	A month view. Events and task deadlines found in meetings appear on their dates. Download `.ics` files or add to Google Calendar.
Minutes of Meeting	Generate, edit, accept, download (Word / PDF / Markdown) and email the minutes.
Follow-ups	Suggested reminders and messages. You review, edit and confirm before anything is sent.
NLP Lab	A "look under the hood" page: try the NLP live, see accuracy numbers, see how often the AI model was needed, and tune the settings.
Settings	Connect free AI services, **save team voices (Voices tab)**, set confidence thresholds, set up email, back up and restore data.

4. How to use it, step by step

4.1 Your first meeting (no keys needed)

1. Open the app and click **Load sample meetings** on the dashboard. Three realistic meetings are analysed in about a minute. Explore every page with them.

4.2 Adding your own meeting

1. Click **New Meeting**.
2. Drag in a file. Supported: audio (mp3, wav, m4a, ogg, flac...), video (mp4, mov, mkv...), or text (txt, vtt, srt, docx, pdf).

- Audio/video needs a speech engine: add a free **Groq key** in Settings (see section 14).
- Text transcripts work with no keys at all.

1. Optionally type a title, date and tags. Press **Analyse meeting**.
2. Watch the progress steps. When it finishes, click **Open meeting**.

A text transcript looks best like this (all of it is optional, but it helps):

```
Meeting: Weekly Product Sync
Date: 2026-10-14
Attendees: Priya Nair, Rahul Mehta

[00:05] Priya Nair: Let's start with the launch date.
[00:20] Rahul Mehta: I'll finish the payment testing by next Friday.
```

4.3 Reviewing and fixing action items

1. Open the meeting, go to **Action items**.
2. Click any cell to edit the task, owner, deadline, status or priority.
3. Press **Save changes**. Tick **Delete** on a row to remove it.
4. Items marked **Needs review** were uncertain - please check them.
5. Click *Evidence for each task* to see the exact words that were said.

4.4 Asking questions

1. Open **Ask MeetingAI**. Choose *All meetings* or one meeting.
2. Type a question or click a suggestion. Examples:

- "What did we decide about the launch date?"
- "Who is responsible for the security questionnaire?"
- "What deadlines were discussed?"
- "What tasks are still pending?"

1. Open **Sources** under an answer to see the meeting, time and speaker. If the answer is not in your meetings, MeetingAI tells you instead of guessing.

4.5 Minutes of Meeting and email

1. Open **Minutes of Meeting**, pick a meeting, click **Generate minutes**.
2. Edit the text in the **Edit** tab. Check the **Preview** tab.

3. Click **Accept minutes**. This unlocks the **Export & email** tab.
4. Download as Word, PDF or Markdown - or add recipients (comma-separated emails), review the email preview, tick the confirmation box, and press **Send minutes**.
5. If email is not set up, click **Open in my mail app instead**.

4.6 Calendar

Events ("demo on Thursday at 11 AM") and task deadlines are placed automatically. Use the arrows to change months. Under the grid you get an agenda list with **Add to Google** buttons and a `.ics` download that works with Outlook, Apple Calendar and Google Calendar. You can also add events by hand.

4.7 Follow-ups

Open **Follow-ups**. Each card is a draft: a recap, a reminder for a person with pending tasks, a request for owners for unassigned tasks, or a reminder for an upcoming event. Edit the text, add email addresses, and either **Send now** (after ticking the confirmation) or **Open in mail app**. Press **Improve with AI** to polish the tone (facts stay the same).

4.8 Showing who said what (speaker names)

Audio has no names in it - a recording is just sound. MeetingAI uses **pyannote** to work out *which voice is which* ("Speaker 1", "Speaker 2"...), and then helps you put real names on them in three ways:

1. **Voice profiles (automatic, best).** Go to **Settings -> Voices**, type a person's name and upload (or record) a 5-30 second clip of only that person talking. pyannoteAI turns it into a *voiceprint*. From then on, every new recording that contains that voice shows **their name** automatically. Voices it does not know stay "Speaker N".
2. **Suggestions from the conversation.** Open the meeting -> **Transcript** -> **Who is who?**. MeetingAI suggests names from what people said - "Hi, I'm Priya", "Rahul, can you give the update?" (then the next voice is Rahul), "Thanks Rahul". Every suggestion shows its evidence and confidence. You can also press **Ask AI for suggestions**; the AI must quote the exact words that prove a name.
3. **Type it yourself.** In the same panel, play the short audio sample under each speaker, type the name, and press **Apply names**. Task owners and the search index are updated too.

Tip: after naming someone, press **Remember this voice** to save their voiceprint, so you never have to name them again.

---

5. How it works inside - the story of one meeting

Imagine you upload a recording. Here is what happens, in order:

```
1. UPLOAD        you add a file
       |
2. FFmpeg        converts any audio/video into small, clean audio chunks
       |
3. SPEECH-TO-TEXT   Whisper turns the audio into text with WORD timestamps
       |
3b. WHO SPOKE WHEN  pyannote splits the audio into voices (Speaker 1, 2, 3...)
       |           saved voice profiles turn voices into NAMES (Priya, Rahul...)
       |           every word is given to the speaker who was talking
       |
4. SAVE          transcript is stored in the SQLite database
       |
5. CLASSICAL NLP    rules + TF-IDF + Naive Bayes + date parser look at every sentence:
       |           "Is this a task? a decision? a question? small talk?"
       |
6. CONFIDENCE GATEWAY   "Am I sure enough?"
       |            sure   -> accept
```

```
        |            unsure -> send this sentence to the AI model (LLM)
        |
 7. LLM (optional)   Groq / Gemini / OpenRouter double-check unsure items
        |           and write the summary
        |
 8. VALIDATION    every AI answer is checked against the real transcript
        |           (quote must exist, owner must be a real participant,
        |            dates are computed by code, never by the AI)
        |
 9. SAVE + INDEX  tasks, decisions, events, summary -> SQLite
        |           transcript chunks -> ChromaDB (for search by meaning)
        |
10. READY         dashboard, calendar, tracker, chat, search all update
```

If you have no AI key, steps 7 is skipped. The app still works using classical NLP and clearly marks uncertain items **Needs review**.

6. The technologies we used - and why

6.1 The big picture

Job	Technology	Why we chose it
The website you see	**Streamlit**	You write the whole interface in Python. No separate website code. Free hosting on Streamlit Community Cloud.
The look and feel	Custom **CSS** inside Streamlit	Streamlit's default look is plain. CSS gives us the dark navy theme, glass cards, gradients and animations.
Charts	**Plotly**	Interactive charts that can be themed to match.
Saving structured data	**SQLite**	A full database that is just a file. No server to install or pay for. Part of Python.
Keyword search	**SQLite FTS5**	Built-in fast full-text search (with stemming: "deploy" finds "deployment").
Search by meaning	**ChromaDB** + **ONNX MiniLM embeddings**	A vector database that runs inside the app. The embedding model is small and does not need PyTorch, so it fits on free hosting.
Understanding sentences (classical NLP)	**scikit-learn** (TF-IDF, Naive Bayes, Logistic Regression), **regular expressions**, **rapidfuzz**	Fast, explainable, free, offline. They give a confidence score and show their reasoning.
Dates like "next Friday"	Our own **date resolver**	Deterministic code, tested with unit tests. The AI is never allowed to invent calendar dates.
Audio/video conversion	**FFmpeg**	The standard tool for it.
Speech to text	**Groq Whisper (large-v3-turbo)**, local Whisper (optional)	Whisper is the best open speech model; Groq runs it very fast on a free tier.
Who is speaking	**pyannote** - hosted **pyannoteAI** API (works on free hosting, can recognise saved voices by name) or **pyannote.audio** run locally (needs PyTorch + a Hugging Face token); **AssemblyAI** as an alternative	pyannote is the best-known open speaker-diarization toolkit. The hosted API gives its quality without PyTorch's ~2 GB, which the free cloud tier cannot hold.
AI language models	**Groq (Llama)**, **Google Gemini**, **OpenRouter free models**	All have free tiers. If one is busy or rate-limited, the app automatically tries the next.
Word / PDF export	**python-docx**, **ReportLab**	Pure Python, no Word installation needed.
Email	**SMTP** (Gmail app password, Brevo, Outlook...)	Works with any email provider; no paid service required.

Job	Technology	Why we chose it
Calendar	In-app calendar + **.ics** + Google Calendar link	No login or permissions needed; the user confirms by clicking.

6.2 Why Streamlit?

Streamlit turns Python scripts into web apps. For a data/AI project this means one language, fast development, and one-click deployment. Its weakness is a plain default design, which is why we added a custom theme.

6.3 Why SQLite and ChromaDB together?

They do different jobs, like a filing cabinet and a librarian:

- **SQLite = the filing cabinet.** It stores the real facts: transcripts, tasks, decisions, chat history. It answers exact questions: "all pending tasks for Priya".
- **ChromaDB = the librarian who understands meaning.** It stores each piece of transcript as a list of numbers (an *embedding*). Pieces with similar meaning get similar numbers. So "What problems delayed the project?" can find "Testing took longer than expected, so we missed the milestone" - even though they share no important words.

ChromaDB is a *derived* index: if it is ever lost, it is rebuilt from SQLite with one button (Settings -> Rebuild semantic index).

6.4 Why free LLMs through a "router"?

Free tiers have limits (requests per minute, tokens per day). The app's router (`core/llm.py`) tries Groq first; if it is rate-limited or errors, it tries Gemini, then OpenRouter. For very long meetings it prefers Gemini because it can read much more text at once. The app only asks the LLM for what it needs (one call for the summary, one batched call for uncertain items), so free limits last.

6.5 Why pyannote - and why through a hosted API?

pyannote is the leading open-source toolkit for *speaker diarization* ("who spoke when"). Run on your own machine it needs **PyTorch** (about 2 GB of libraries), a free **Hugging Face token**, and a model licence you must accept. That is fine on a laptop but **does not fit on free hosting** (about 1 GB of memory).

So MeetingAI supports two ways of using pyannote:

Way	Works on Streamlit Cloud?	Gives names?
pyannoteAI hosted API (`PYANNOTEAI_API_KEY`)	Yes	Yes - with saved voice profiles
pyannote.audio on your own PC (`HF_TOKEN` + `pip install pyannote.audio`)	No	Only "Speaker 1/2/3" (you add names)

The app picks automatically (Settings -> AI & speech -> Speaker engine). If a speaker service fails or runs out of credits, the transcript is still saved - with a single speaker and a clear warning.

6.6 Why no PyTorch?

PyTorch is large (hundreds of MB). Free hosting gives about 1 GB of memory. The ONNX version of the MiniLM embedding model is small and fast, so semantic search works on the free tier.

---

7. The NLP part explained simply

NLP (Natural Language Processing) means teaching computers to work with human language. Nowadays big language models (LLMs) are themselves NLP. MeetingAI uses *both* old-school ("classical") NLP and modern LLMs, each for what it is best at.

7.1 Every feature is an NLP task

Feature	NLP task
Speech to text	Automatic speech recognition (ASR)
"Speaker A / Speaker B"	Speaker diarization (pyannote)
"Speaker B" -> "Priya"	Speaker identification (voiceprints) + name inference from the text (self-introductions, being addressed)
Summary	Abstractive / extractive summarisation
Finding tasks and decisions	Sentence classification, information extraction
"Priya will send it" -> owner = Priya	Named entities, semantic roles
"I'll do it" -> who is "I"?	Coreference (here: the speaker)
"next Friday" -> 23 Oct	Temporal expression normalisation
Search by meaning	Dense retrieval with sentence embeddings
Chatbot	Retrieval-augmented generation (RAG)
"Not found" answers	Answerability detection

7.2 The classical NLP tier, in plain words

For every sentence, MeetingAI asks several simple "experts":

1. **Cleaning.** Remove fillers ("um", "you know") and split text into sentences.
2. **Rule expert.** Looks for telltale patterns. "*X will send...*", "*can you prepare...*", "*action item*", "*we decided*", "*let's go with*". Each pattern has a weight. It also spots hedging words ("maybe", "might", "consider"), which signal a suggestion rather than a commitment.
3. **Statistics expert (TF-IDF + Naive Bayes).** *TF-IDF* turns a sentence into numbers that show which words matter. *Naive Bayes* is a simple, fast classifier that looks at those numbers and says: "I think this is 85% an action, 10% discussion...". It was trained on a seed set of 610 example sentences (action / decision / question / discussion).
4. **Fusion.** The two experts vote: final score = 50% statistics + 50% rules. If the sentence is hedged, the score is reduced by 20%.
5. **Owner finder.** Patterns detect who is responsible: a name before "will", "Priya, can you...", "assigned to...", "I will" (= the speaker), "we/someone" (= no single owner -> Unassigned). Each guess has a confidence.
6. **Deadline finder.** Our date resolver finds phrases ("by Monday", "end of the month", "in two weeks", "25th October", "tomorrow at 3 PM") and converts them using the **meeting's date** as the starting point. Vague phrases like "next week" or "soon" are kept as text, and the date stays **Not specified** - we refuse to pretend to be precise.
7. **Event finder.** A sentence with a meeting-like word (call, demo, review, sync...) plus a clear date becomes a calendar event.

7.3 Where the LLM comes in

LLMs are brilliant at nuance but can be wrong, slow, and rate-limited. So we use them for:

- writing the **summary** (a generative job),
- **double-checking** sentences the classical tier is unsure about,
- an optional **second look** at the whole transcript to catch tasks the rules missed.

And we never trust them blindly (section 9).

7.4 Speaker diarization and speaker names, step by step

1 **Word timestamps.** Whisper returns every word with its start and end time.

2 **Diarization.** pyannote listens to the whole recording and returns *turns*: `Speaker 1: 0.0-3.2 s`, `Speaker 2: 3.4-9.0 s`...

3 **Alignment** (`core/diarize.py` -> `assign_speakers`). Each word is handed to the speaker whose turn contains the word's midpoint. A sentence is split exactly where the speaker changes. One-word interruptions ("yeah") do not break a turn in pieces.

4 **Voice matching (optional).** If you saved voice profiles, MeetingAI asks pyannoteAI's *identify* job to compare each voice with the voiceprints. A voice that matches well enough (default strictness 60 out of 100) gets the **name**; the rest remain Speaker N.

5 **Name suggestions** (`core/speakers.py`) for anonymous voices, from the transcript:

- *self-introduction* ("I'm Priya") - strong evidence,
- *being addressed* ("Rahul, can you..." makes the next voice Rahul; "Thanks Rahul" makes the previous voice Rahul) - medium,
- *elimination* (one unnamed voice and one unused attendee) - weak,
- *LLM* (optional) - must quote exact words and the name must be an attendee or appear in the transcript.

1 **You confirm.** Suggestions are never applied silently. **Apply names** renames the transcript, task owners, decisions and the attendee list, and refreshes the search index.

---

8. The confidence gateway - the heart of the project

The gateway is a checkpoint between the classical tier and the LLM.

For each candidate task or decision it asks:

Question	Default threshold	If "no"
Is the classifier confident enough this is a task/decision?	score >= 0.80	send to LLM
Is it clearly ahead of the other categories?	margin >= 0.15	send to LLM
Is the owner reliably known?	owner confidence >= 0.85	send to LLM
Was the date found in the same sentence?	-	send to LLM if found only in the next sentence

Every decision is saved with a **reason code** and a plain-English message, for example:

> *LLM fallback used: confidence 0.63 is below the threshold 0.80.*

Reason codes include `CLASSICAL_ACCEPTED`, `LOW_CLASSIFICATION_CONFIDENCE`, `LOW_CLASS_MARGIN`, `OWNER_UNCERTAIN`, `DEADLINE_FROM_NEIGHBOUR`, `HEDGED_LANGUAGE`, `MISSING_EVIDENCE`, `CONFLICTING_EXTRACTIONS`, `LLM_CONFIRMED`, `LLM_REJECTED`, `NO_LLM_AVAILABLE`.

A worked example:

> "Priya will send the project report by Monday." -> rules: strong (0.92), Naive Bayes: 0.98 -> fused ~ 0.98. Owner "Priya" is a known speaker (0.95). Date "Monday" after "by" (0.95). **Accepted without the LLM.** > > "Please book the conference room." -> looks like a task (0.87) but **who**? No owner -> owner confidence 0.5 -> `OWNER_UNCERTAIN` -> sent to the LLM; if the LLM cannot find an owner either, the task is saved as **Unassigned**. > > "Maybe we should consider hiring another developer." -> hedging words -> low score -> not a firm task -> never enters the tracker.

You can see all of this live in **NLP Lab** and change the thresholds in **Settings -> Confidence gateway**. Stricter thresholds mean more LLM calls but fewer mistakes; the *Threshold simulator* chart in NLP Lab shows the trade-off.

If two sources disagree (e.g. classical says owner = A, LLM says owner = B), the item is marked `CONFLICTING_EXTRACTIONS` and **Needs review**. If both agree, confidence rises to 0.95 (the label becomes "NLP + LLM agree").

---

9. How MeetingAI avoids making things up

AI models sometimes "hallucinate" - they produce confident nonsense. MeetingAI uses several layers of protection:

1. **Verbatim quotes.** The LLM must give an exact quote for every task, decision and event. Code checks that the quote really occurs in the transcript (fuzzy match). No quote -> the item is dropped (`MISSING_EVIDENCE`).
2. **Owners must exist.** An owner proposed by the LLM must be a participant or appear in the transcript. Otherwise the owner becomes "Unassigned" and the item is flagged for review.
3. **Dates are computed by code.** The LLM only copies the phrase ("next Friday"). Our deterministic resolver converts it, relative to the meeting date. The phrase must actually appear near the quote.
4. **Answers cite sources.** Chat answers must cite evidence ids like [E1]. Citations that do not exist are ignored.
5. **A "not found" gate.** If no retrieved passage is relevant enough (similarity score or word overlap), the app answers "I couldn't find that" **without even calling the LLM**.
6. **Support check.** After an answer is written, we measure how much of it is backed by the cited evidence. Low overlap or numbers that are not in the evidence produce a visible warning.
7. **Honest labels.** The UI shows who produced each item (Classical NLP / LLM / both agree), its confidence, and whether it needs review.

---

10. Search and the chatbot - how they find answers

10.1 Two ways of searching, fused

For every question MeetingAI searches in two ways at once:

- **Semantic search (ChromaDB):** compares the *meaning* of the question to chunks of the transcript.
- **Keyword search (SQLite FTS5):** looks for the actual words (with stemming).

Then **Reciprocal Rank Fusion** merges the two ranked lists: a passage that ranks well in both rises to the top.

10.2 The query router

Different questions need different tools. The router reads the question and decides:

Question type	What is used
"Who is responsible...", "deadlines", "pending tasks"	The task table in SQLite (plus passages)
"What did we decide..."	The decisions table (plus passages)
"upcoming events", "schedule"	The events table
"summarise...", "compare across..."	Saved summaries
"last meeting", "last three meetings", "on October 10", a meeting name	Limits the search to those meetings
A phrase in quotes	Exact phrase search
Everything else	Hybrid semantic + keyword search

10.3 Writing the answer

The best evidence (up to six passages and the relevant structured records) is given to the LLM with strict instructions: answer only from this, cite it, and say what is missing. If no LLM is configured, MeetingAI shows the retrieved facts directly (a "direct retrieval" answer) - still useful, never invented.

Chat sessions, messages, sources and metadata are saved, so you can continue a conversation days later.

---

11. Where your data lives (and privacy)

What	Where
Meetings, transcripts, summaries, tasks, decisions, events, MoM, chats, follow-ups, logs	SQLite file: `data/meetingai.db`
Search vectors	ChromaDB folder: `data/chroma/`
Uploaded recordings	`data/uploads/`
API keys	Never stored by the app. Environment variables, `secrets.toml`, or typed in Settings (kept in the browser session only).

What leaves your computer? Only if you configure AI services:

- Audio you transcribe goes to Groq (text) and, if speaker recognition is on, to pyannoteAI (or AssemblyAI). Local pyannote keeps audio on your machine.
- **Voiceprints** (the voice fingerprints) are stored in your own database. pyannoteAI does not keep them for you. Only create a voice profile for someone who agreed to it, and delete it from Settings -> Voices any time.
- Transcript text goes to Groq / Gemini / OpenRouter when the LLM is used. You can turn this off in *New Meeting* ("Use AI model" switch) - then everything stays local (apart from audio transcription).

On free cloud hosting, the disk is temporary: data can vanish when the app restarts. Use **Settings -> Data & backup** to download a backup `.zip` regularly, and restore it after a restart.

Public deployments: anyone with the link would share the same data and your API quotas. Set `APP_PASSWORD` in the secrets to require a password.

11.1 Database tables

`meetings`, `segments` (+ `segments_fts` search index), `voiceprints`, `summaries`, `decisions`, `tasks`, `events`, `mom`, `chat_sessions`, `chat_messages`, `followups`, `analysis_log` (gateway decisions), `email_log`, `settings`.

---

12. The project folder, file by file

```
nlp+agentic ai/
|- app.py                  App entry point: page setup, login gate, sidebar, page router
|- requirements.txt        Python packages to install
|- packages.txt            System packages for Streamlit Cloud (ffmpeg)
|- README.md               Short quick-start
|- .streamlit/
|   |- config.toml         Dark theme colours and upload limit
|   `- secrets.toml.example   Template for API keys (copy to secrets.toml)
|- core/                   The "brain" (no UI code)
|   |- config.py           Paths, defaults, secret lookup
|   |- db.py               SQLite tables and every read/write function
|   |- parsing.py          Reads txt / vtt / srt / docx / pdf transcripts
|   |- stt.py              Speech-to-text (FFmpeg, Groq, AssemblyAI, local Whisper) + word timestamps
|   |- diarize.py          pyannote: who spoke when, voice profiles, aligning words to speakers
|   |- speakers.py         Name suggestions for anonymous speakers and applying names
|   |- dates.py            Finds and converts "next Friday", "end of month"...
|   |- seed_data.py        Training and test sentences for the classifier
|   |- nlp.py              Cleaning, rules, Naive Bayes, owner / deadline / event extraction
|   |- gateway.py          The confidence gateway and reason codes
|   |- llm.py              Free-LLM router with fallback
|   |- llm_tasks.py        The prompts given to the LLM and how answers are parsed
|   |- pipeline.py         Orchestrates everything for one meeting + validation
|   |- rag.py              ChromaDB index, hybrid search, query router, grounded answers
|   |- mom.py              Minutes of Meeting (Markdown / HTML / Word / PDF)
|   |- mailer.py           Email sending (SMTP)
|   |- ics.py              Calendar files and Google Calendar links
|   |- followups.py        Draft reminders and messages
|   `- demo.py             Three sample meetings
|- ui/
|   |- theme.py            All the CSS (colours, cards, animations)
|   |- components.py       Reusable HTML pieces (cards, chips, transcript, calendar grid)
|   |- charts.py           Themed Plotly charts
|   `- nav.py              Navigation helpers
|- views/                  One file per page
|   |- dashboard.py  upload.py  library.py  meeting.py  chat.py  search.py
|   |- tracker.py  calendar.py  mom.py  followups.py  nlplab.py  settings.py
|   |- _tasks.py           Editable task table used by two pages
|   |- _speakers.py        The 'Who is who?' panel (naming speakers, saving voices)
|   `- _email.py           'Connect your email' panel
|- tests/                  79 automated tests
`- docs/                   This guide
```

---

13. Setting it up on your own computer

You need **Python 3.10 or newer** (3.11 recommended).

Windows (PowerShell)

```
cd "C:\Users\Nidhi\Desktop\SEM 3\nlp+agentic ai"
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
winget install Gyan.FFmpeg
streamlit run app.py
```

(FFmpeg is only needed for audio/video. Close and reopen PowerShell after installing it.)

Mac / Linux

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
brew install ffmpeg        # or: sudo apt install ffmpeg
streamlit run app.py
```

The app opens at **http://localhost:8501**. The first time you analyse a meeting, the small embedding model (about 80 MB) downloads automatically.

Optional: run pyannote on your own PC (no hosted API)

```
pip install -r requirements-local.txt
```

Then create a free token at huggingface.co/settings/tokens, **accept the model terms** on the pages huggingface.co/pyannote/speaker-diarization-3.1 and huggingface.co/pyannote/segmentation-3.0, and set `HF_TOKEN`. Local pyannote gives "Speaker 1/2/3" labels (naming is then done in the **Who is who?** panel). It is slow on CPU and cannot run on the free Streamlit Cloud tier.

Adding keys permanently

Copy `.streamlit/secrets.toml.example` to `.streamlit/secrets.toml` and fill in what you have. This file is ignored by git, so it will not be uploaded.

---

14. Getting the free API keys

You do not need all of them. **The best minimum is Groq** (speech + AI). Add Gemini for long meetings.

Service	What for	How to get a key
Groq	Whisper speech-to-text and Llama AI models	Go to console.groq.com -> sign in -> **API Keys** -> **Create API key**. Free tier, no card.
Google Gemini	AI for long meetings (backup)	Go to aistudio.google.com/apikey -> **Create API key**. Free tier.
OpenRouter (optional)	More free models as a backup	openrouter.ai -> **Keys**. Choose models ending in `:free`.
pyannoteAI (recommended for speaker names)	pyannote speaker separation and voice recognition	dashboard.pyannote.ai -> sign up -> **API Keys** -> **Create new key**. Copy it immediately (it is shown once). Check the dashboard for current free credits and pricing.
Hugging Face token (local pyannote only)	Running pyannote.audio on your own PC	huggingface.co/settings/tokens, and accept the pyannote model terms.
AssemblyAI (optional)	Speaker identification for audio	assemblyai.com -> sign up -> copy the API key from the dashboard. Free credits.
Gmail app password (optional)	Sending emails	Google Account -> **Security** -> turn on **2-Step Verification** -> **App passwords** -> create one named "MeetingAI". Use host `smtp.gmail.com`, port `587`, your Gmail address as user, and the 16-character app password.

Free-tier limits and model names change over time. If a model name stops working, set `GROQ_MODEL` or `GEMINI_MODEL` in the secrets (see `secrets.toml.example`). **Settings -> Test connections** tells you straight away if a key works.

---

15. Putting it on the internet (Streamlit Community Cloud)

Streamlit Community Cloud is free. It runs your app straight from a GitHub repository.

Step 1 - Put the code on GitHub

1. Create a free account at github.com if you do not have one.
2. Click **New repository**. Name it `meetingai`. Keep it **Public** - that is the simplest option (private repos also work, but you must give Streamlit permission to read them). Do **not** add a README (the project already has one).
3. In PowerShell, inside the project folder:

```
git add .
git commit -m "MeetingAI"
git branch -M main
git remote add origin https://github.com/<your-username>/meetingai.git
git push -u origin main
```

`.gitignore` makes sure your keys, database and recordings are **not** uploaded.

Step 2 - Create the app

1 Go to **share.streamlit.io** and sign in with GitHub.

2 Click **Create app** -> **Deploy a public app from GitHub**.

3 Choose the repository `meetingai`, branch `main`, main file path `app.py`.

4 Click **Advanced settings**:

- Python version: **3.11** or **3.12**.
- **Secrets**: paste your keys in this format:

```
GROQ_API_KEY = "gsk_..."
GEMINI_API_KEY = "AIza..."
PYANNOTEAI_API_KEY = "..."   # speaker names (pyannote)
APP_PASSWORD = "choose-a-strong-password"
```

1 Click **Deploy**. The first build takes a few minutes. `packages.txt` installs FFmpeg automatically.

Step 3 - Use it

You get a link like `https://your-name-meetingai.streamlit.app`. Open it, enter your password, load the samples or upload a meeting.

Things to know about free hosting

- **Memory is limited (about 1 GB).** MeetingAI is designed to fit (ONNX embeddings, no PyTorch), but very large uploads or many simultaneous users may strain it. Use Groq for transcription rather than local Whisper.
- **The app sleeps** after a period of no visits; the first visit wakes it up (about a minute).
- **The disk is temporary.** When the app restarts, meetings may be gone. Download a backup from *Settings -> Data & backup* and restore it when needed.
- **Everyone with the link sees the same data.** Keep `APP_PASSWORD` set.
- To update the app, just `git push` again - Streamlit redeploys by itself.

---

16. Other free places to host it

Option	Good for	Watch out for
Streamlit Community Cloud	Easiest. Connect GitHub and deploy.	~1 GB RAM, temporary disk, sleeps when idle.
Hugging Face Spaces (Docker or Streamlit SDK)	More RAM (~16 GB on the free CPU tier); good if you want local Whisper.	Disk also resets unless you add storage; slower CPU.
Render (free web service)	Simple Docker deploys.	Sleeps after ~15 min idle; slow wake-up.
Google Cloud Run	Scales to zero; generous free tier.	Needs a billing account on file.
Run on your own PC + a tunnel (Cloudflare Tunnel / ngrok)	Full power, your own disk.	Your computer must stay on.

---

17. How we tested it, and what its limits are

17.1 Tests

The project contains **79 automated tests** (run with `python -m pytest tests`), including a fake pyannoteAI server (upload, diarize, identify, voiceprint), word-to-speaker alignment, voice-sample cutting and name suggestions:

- date understanding (weekdays, "end of month", vague phrases, ambiguous dates),
- classical NLP (owners, deadlines, decisions, hedging, events),
- transcript parsing (speaker lines, timestamps, VTT, headers),
- the full pipeline on the sample meetings,
- the LLM path with a **fake LLM** - including proof that invented quotes and invented owners are rejected,
- search, "not found" behaviour, chat answers,
- Minutes export (Word, PDF, HTML), email sending (against a local test mail server), calendar files, audio chunking with FFmpeg,
- every page of the app rendering without errors.

17.2 Accuracy numbers (NLP Lab)

On 42 hand-written test sentences the classical models score roughly: rules ~ 98%, Naive Bayes ~ 76%, logistic regression ~ 81%, fused model ~ 93%. **Please read these honestly:** the training and test sentences were written by the project authors, so the numbers are optimistic. They show the pipeline works - not how it performs on any real company's meetings. That is exactly why the confidence gateway and the LLM safety net exist.

17.3 Known limits

- **Speaker separation and names** for audio need a speaker engine (pyannoteAI key, local pyannote, or AssemblyAI). Without one, audio is a single "Speaker". Voice-based names need saved voice profiles; otherwise use the **Who is who?** panel. Overlapping speech and very short interjections can be mislabelled. Text transcripts with names are fully supported.
- **Live AI services** (Groq, Gemini, OpenRouter, AssemblyAI, **pyannoteAI**) and **real email delivery** need your keys and could not be exercised with real credentials during development; those code paths were tested with fakes and local servers. Use *Settings -> Test connections* after adding keys.
- Free AI tiers have rate limits and change model names over time.
- English is the main supported language.
- Very vague scheduling ("sometime next quarter") is shown as text and not put on the calendar.
- The classical tier is a baseline trained on a small seed set. Real labelled meeting data would improve it.

---

18. Troubleshooting

Problem	Solution
"No speech-to-text engine is configured"	Add a free `GROQ_API_KEY` in Settings, or upload a text transcript.
"FFmpeg was not found"	Install FFmpeg (`winget install Gyan.FFmpeg`, `brew install ffmpeg`, or `packages.txt` on Streamlit Cloud).

Problem	Solution
Answers say "direct retrieval (no AI model)"	No LLM key is configured. Add Groq or Gemini in Settings.
"All LLM providers failed"	Rate limit or wrong key. Try *Settings -> Test connections*, wait a minute, or add a second provider.
Email: "server rejected the login"	For Gmail you must use an **App password**, not your normal password.
Everything says "Speaker" / "Speaker 1"	Add a `PYANNOTEAI_API_KEY` (Settings -> AI & speech) and re-upload; or name voices in Transcript -> **Who is who?**.
"pyannoteAI says a subscription or credits are required (402)"	Your pyannoteAI account has no credits left. Check dashboard.pyannote.ai. The transcript is still saved with one speaker.
A person is labelled with the wrong name	Raise **Match strictness** in Settings -> Voices, or re-create that person's voice profile from a longer, cleaner clip.
Search finds nothing	Check the meeting finished processing; try *Settings -> Rebuild semantic index*.
Meetings disappeared on the hosted app	The free disk is temporary. Restore your backup (Settings -> Data & backup).
First analysis is slow	The embedding model downloads once (about 80 MB).
A task has the wrong owner	Edit it in the tracker. Owner edits clear the "needs review" flag.

19. Ideas for the future

- Train the classifier on real annotated meetings and calibrate its probabilities.
- Automatic speaker naming from introductions.
- Multi-language meetings.
- Direct Google Calendar creation after OAuth sign-in.
- User accounts and per-user data.
- A persistent hosted database for always-on deployments.
- Live (real-time) meeting capture.

20. Glossary

Action item - a task someone agreed to do. **ASR (speech recognition)** - turning audio into text. **Chunk** - a short piece of transcript stored for searching. **Classifier** - a program that puts things into categories. **Confidence** - a number from 0 to 1 showing how sure a model is. **Cosine similarity** - a score of how close in meaning two embeddings are (1 = identical). **Diarization** - working out who spoke when. **Embedding** - a list of numbers that represents the meaning of a text. **FTS5** - SQLite's built-in full-text search. **Gateway (confidence gateway)** - the checkpoint that decides whether classical NLP is trusted or the LLM must double-check. **Hallucination** - when an AI states something false with confidence. **LLM (large language model)** - an AI such as Llama or Gemini that reads and writes text. **Margin** - the gap between the best and second-best guess. **MoM** - Minutes of Meeting. **Naive Bayes** - a simple, fast probability-based classifier. **NLP** - Natural Language Processing: computers working with human language. **RAG (retrieval-augmented generation)** - first *find* relevant text, then let the AI answer using only that text. **RRF (reciprocal rank fusion)** - a way to merge two ranked result lists. **SQLite** - a database stored in a single file. **TF-IDF** - a way of scoring how important a word is in a sentence compared with all others. **Transcript** - the written text of a meeting. **Vector database (ChromaDB)** - a database that finds items by similarity of meaning. **Voiceprint** - a compact numeric fingerprint of a person's voice, used to recognise them in later recordings. **Whisper** - OpenAI's open speech-recognition model.